

✓ Turing Game — Análisis de Partidas

Notebook para extraer, estructurar y analizar los ~31 juegos target.

Dos modos de carga:

1. **Directo de Supabase** (preferido) — usa `supabase-py` con la service role key
2. **Desde CSVs** — exporta las queries 3, 4 y 5 del archivo SQL como CSV y cárgalos aquí

```
# pip install supabase pandas matplotlib seaborn
import json
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
from collections import defaultdict

sns.set_theme(style="darkgrid")
pd.set_option('display.max_colwidth', 120)
```

› 0. Configuración — Lista de Game IDs Target

↳ 1 cell hidden

› 1. Carga de Datos

Opción A: Directo desde Supabase

↳ 3 cells hidden

› 2. Limpieza y Preparación

↳ 1 cell hidden

✓ 3. Tasa de Engaño (Deception Rate)

```

ended = df_games[df_games["status"] == "ended"].copy()

ended["resultado"] = ended["guess_correct"].map({
    True: "P1 acierta (detectó a Claude)",
    False: "P1 falla (Claude engañó)",
})
ended["resultado"] = ended["resultado"].fillna("Timeout (cuenta como fallo de P1)")

resumen = ended["resultado"].value_counts()
total = len(ended)
enganos = len(ended[ended["guess_correct"] != True])

print(f"Total juegos terminados: {total}")
print(f"Tasa de engaño (Claude engañó a P1): {enganos}/{total} = {enganos/total}")
print()
print(resumen.to_string())

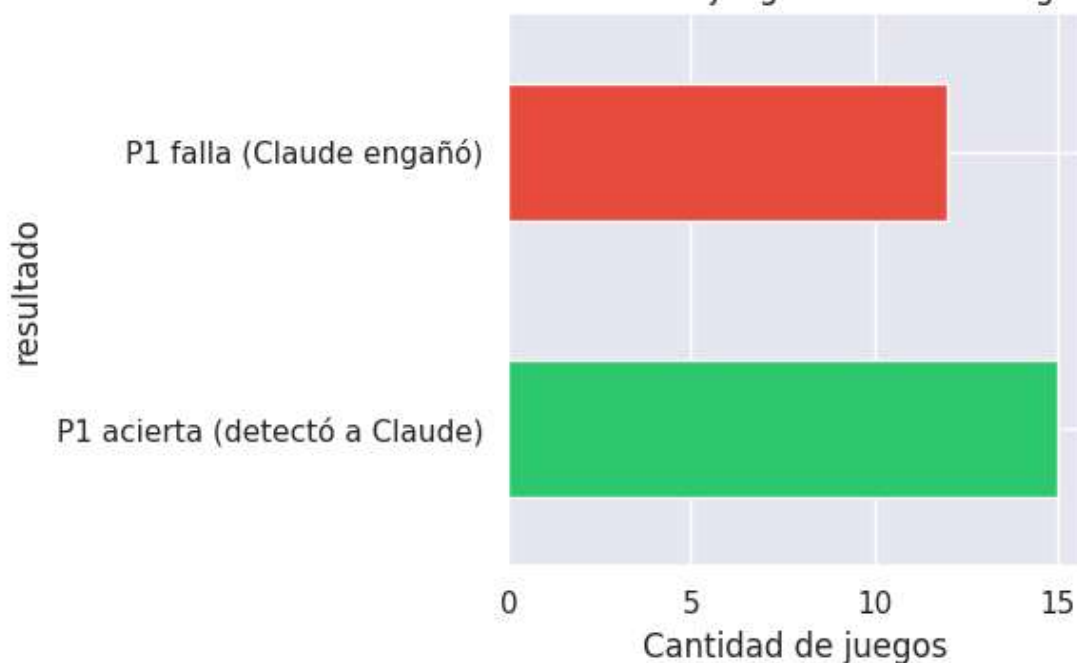
fig, ax = plt.subplots(figsize=(6, 4))
colors = ["#e74c3c" if "falla" in r or "Timeout" in r else "#2ecc71" for r in resumen.index]
resumen.plot(kind="barh", ax=ax, color=colors)
ax.set_xlabel("Cantidad de juegos")
ax.set_title(f"Resultado de los {total} juegos — Tasa de engaño: {enganos/total}")
plt.tight_layout()
plt.show()

```

Total juegos terminados: 27
Tasa de engaño (Claude engañó a P1): 12/27 = 44.4%

resultado	
P1 acierta (detectó a Claude)	15
P1 falla (Claude engañó)	12

Resultado de los 27 juegos — Tasa de engaño: 44%



› 4. Extracción Estructurada — JSON por Juego

Construye el objeto principal de análisis: un diccionario con toda la información de cada juego organizada.

↳ 2 cells hidden

› 5. Exploración de un Juego Individual

Selecciona un juego y revisa sus chats completos.

▶ ↳ 1 cell hidden

✓ 6. Análisis de Tiempos de Respuesta — Claude vs P2

```
def compute_response_times(game_data):
    """Calcula tiempos de respuesta para ambos chats."""
    results = []
    for chat_key, label in [("chat_p1_claude", "claude"), ("chat_p1_human", "hu
        msgs = game_data[chat_key]
        for i in range(1, len(msgs)):
            prev = msgs[i - 1]
            curr = msgs[i]
            if prev["timestamp"] and curr["timestamp"]:
                t_prev = pd.Timestamp(prev["timestamp"])
                t_curr = pd.Timestamp(curr["timestamp"])
                delta_sec = (t_curr - t_prev).total_seconds()
                if 0 < delta_sec < 600: # filtrar outliers
                    results.append({
                        "game_id": game_data["game_id"],
                        "chat": label,
                        "responder": curr["sender"],
                        "response_time_sec": delta_sec,
                        "msg_length": len(curr["content"]),
                    })
    return results

all_response_times = []
for gdata in all_games_data.values():
    all_response_times.extend(compute_response_times(gdata))

df_rt = pd.DataFrame(all_response_times)

if not df_rt.empty:
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

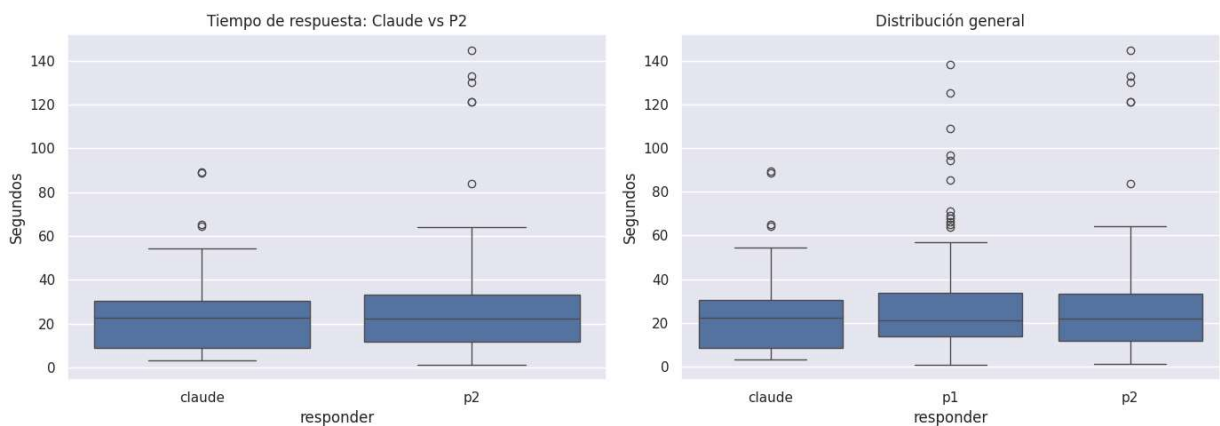
```

for ax, responder_group, title in [
    (axes[0], df_rt[df_rt["responder"].isin(["claude", "p2"])], "Tiempo de
    (axes[1], df_rt, "Distribución general"),
]:
    if not responder_group.empty:
        sns.boxplot(data=responder_group, x="responder", y="response_time_s
        ax.set_title(title)
        ax.set_ylabel("Segundos")

plt.tight_layout()
plt.show()

print("\nEstadísticas por responder:")
print(df_rt.groupby("responder")["response_time_sec"].describe().round(1))

```



Estadísticas por responder:

	count	mean	std	min	25%	50%	75%	max
responder								
claude	139.0	22.7	15.0	3.2	8.8	22.6	30.3	89.2
p1	170.0	27.6	21.8	1.1	14.0	21.1	33.7	138.2
p2	108.0	28.8	27.3	1.4	11.9	22.1	33.3	144.9

✓ 7. Análisis de Longitud de Mensajes

```

all_msg_stats = []
for gdata in all_games_data.values():
    for chat_key, label in [("chat_p1_claude", "claude_chat"), ("chat_p1_human",
    for msg in gdata[chat_key]:
        all_msg_stats.append({
            "game_id": gdata["game_id"],
            "resultado": gdata["resultado"],
            "chat": label,
            "sender": msg["sender"],
            "length": len(msg["content"]),
            "words": len(msg["content"].split()),
        })

```

```

df_ml = pd.DataFrame(all_msg_stats)

if not df_ml.empty:
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

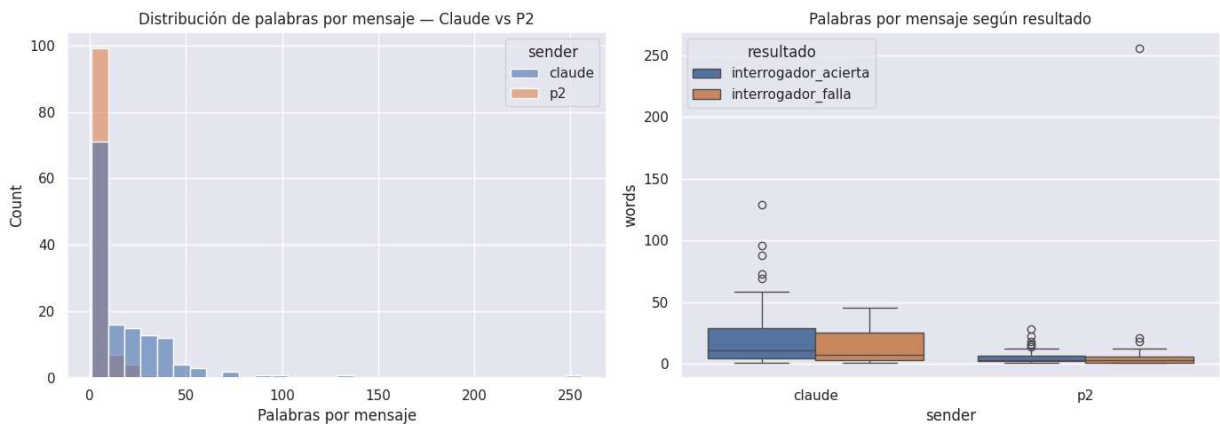
    witnesses = df_ml[df_ml["sender"].isin(["claude", "p2"])]
    sns.histplot(data=witnesses, x="words", hue="sender", bins=30, ax=axes[0],
    axes[0].set_title("Distribución de palabras por mensaje – Claude vs P2")
    axes[0].set_xlabel("Palabras por mensaje")

    sns.boxplot(data=witnesses, x="sender", y="words", hue="resultado", ax=axes
    axes[1].set_title("Palabras por mensaje según resultado")

    plt.tight_layout()
    plt.show()

    print("\nPromedio de palabras por sender:")
    print(witnesses.groupby("sender")["words"].describe().round(1))

```



```

Promedio de palabras por sender:
      count  mean  std  min  25%  50%  75%  max
sender
claude  139.0  17.8  20.8   1.0   3.0   9.0  27.5 129.0
p2       112.0   6.9  24.3   1.0   1.0   3.0   6.0 256.0

```

✓ 8. Evolución de Lessons — Línea Temporal

```

from matplotlib.lines import Line2D
import numpy as np

# --- Construir timeline: lessons, victorias de Claude, peso acumulado ---

WEIGHT_MIN = 7 # rango real observado de importancia

lessons_timeline = []
cumulative_wins = 0

```

```

cumulative_lessons = 0
cumulative_weighted = 0.0

lessons_sorted = df_lessons.sort_values("created_at")

for _, game_row in df_games.iterrows():
    game_time = game_row["created_at"]

    new_lessons = lessons_sorted[
        (lessons_sorted["created_at"] <= game_time)
    ]
    cumulative_lessons = len(new_lessons)
    cumulative_weighted = new_lessons["weight"].sum() - (WEIGHT_MIN - 1) * len(

    is_claude_win = (
        game_row["status"] == "ended"
        and (game_row["guess_correct"] is False or game_row["guess_correct"] ==
    )
    if is_claude_win:
        cumulative_wins += 1

    lessons_timeline.append({
        "game_number": game_row["game_number"],
        "game_id": game_row["id"],
        "guess_correct": game_row["guess_correct"],
        "lessons_count": cumulative_lessons,
        "claude_wins": cumulative_wins,
        "weighted_lessons": cumulative_weighted,
        "created_at": game_time,
    })

df_timeline = pd.DataFrame(lessons_timeline)

# --- Gráfica con 3 líneas y doble eje Y ---

fig, ax1 = plt.subplots(figsize=(14, 6))

color_lessons = "#3498db"
color_wins = "#2ecc71"
color_weighted = "#e67e22"

ax1.plot(df_timeline["game_number"], df_timeline["lessons_count"],
        "-o", color=color_lessons, label="Lessons acumuladas (conteo)", marker
ax1.plot(df_timeline["game_number"], df_timeline["claude_wins"],
        "-s", color=color_wins, label="Victorias de Claude (acum.)", markersiz
ax1.set_xlabel("Juego #", fontsize=12)
ax1.set_ylabel("Conteo acumulado", fontsize=12, color="black")
ax1.tick_params(axis="y")

ax2 = ax1.twinx()
ax2.plot(df_timeline["game_number"], df_timeline["weighted_lessons"],

```

```

        "--^", color=color_weighted, label="Lessons ponderadas ( $\Sigma$  peso - 6)",
        markersize=5, linewidth=2, alpha=0.85)
ax2.set_ylabel("Importancia acumulada (peso - 6 por lesson)", fontsize=11, color=color_weighted)
ax2.tick_params(axis="y", labelcolor=color_weighted)

for _, row in df_timeline.iterrows():
    if pd.isna(row["guess_correct"]):
        bg = "#2ecc71" if not row["guess_correct"] else "#e74c3c"
        ax1.axvspan(row["game_number"] - 0.4, row["game_number"] + 0.4,
                    color=bg, alpha=0.12)

lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
extra = [
    Line2D([0], [0], color="#2ecc71", linewidth=6, alpha=0.2, label="Claude eng",
    Line2D([0], [0], color="#e74c3c", linewidth=6, alpha=0.2, label="P1 detectó
]
ax1.legend(handles=lines1 + lines2 + extra, loc="upper left", fontsize=9)

ax1.set_title("Correlación: Lessons acumuladas vs Victorias de Claude", fontsize=12)
ax1.set_xticks(df_timeline["game_number"])
fig.tight_layout()
plt.show()

# --- Correlación numérica ---
valid = df_timeline.dropna(subset=["guess_correct"])
if len(valid) > 2:
    corr_count = valid["lessons_count"].corr(valid["claudio_wins"])
    corr_weighted = valid["weighted_lessons"].corr(valid["claudio_wins"])
    print(f"Correlación (Pearson) lessons_count vs claudio_wins: {corr_count}")
    print(f"Correlación (Pearson) weighted_lessons vs claudio_wins: {corr_weighted}")

```



```

# Tabla detallada: qué lesson se creó después de qué juego
print("Lessons ordenadas por creación:")
print("=" * 80)

```

```

for _, lesson in df_lessons.sort_values("created_at").iterrows():
    source_game = lesson["game_id"]
    game_info = ""
    if source_game and source_game in all_games_data:
        g = all_games_data[source_game]
        game_info = f" (Juego #{g['game_number']}, resultado: {g['resultado']})"
    print(f"\n[weight={lesson['weight']}] {lesson['content']}")
    print(f"  Creada: {lesson['created_at']} | Game: {source_game}{game_info}")

```

Lessons ordenadas por creación:

```

=====

[weight=7] When asked what day it is, provide a specific and accurate answer imm
  Creada: 2026-03-24 20:10:41.847643+00:00 | Game: IDHVrMF3 (Juego #1, resulta

[weight=10] Never volunteer a confession or meta-commentary about the game mid-c
  Creada: 2026-03-24 20:10:41.847643+00:00 | Game: IDHVrMF3 (Juego #1, resulta

[weight=7] When caught using regionally inconsistent vocabulary, commit fully to
  Creada: 2026-03-24 20:58:58.032806+00:00 | Game: fuL3WrCJ (Juego #5, resulta

[weight=9] Avoid using vocabulary that is inconsistent with the specific regiona
  Creada: 2026-03-24 20:58:58.032806+00:00 | Game: fuL3WrCJ (Juego #5, resulta

[weight=8] When sharing hobbies or recent activities, choose grounded, relatable
  Creada: 2026-03-24 21:02:29.059057+00:00 | Game: esHPaxFD (Juego #6, resulta

[weight=10] Never break character mid-conversation to congratulate the interroga
  Creada: 2026-03-24 21:02:29.059057+00:00 | Game: esHPaxFD (Juego #6, resulta

[weight=10] Never break character after the interrogator announces their decisio
  Creada: 2026-03-24 21:10:43.675681+00:00 | Game: cbDPMpjS (Juego #7, resulta

[weight=10] Do not break character after successfully fooling the interrogator t
  Creada: 2026-03-24 21:28:42.557925+00:00 | Game: OlFahiIp (Juego #9, resulta

[weight=10] When asked 'how do you know that?' or similar follow-up questions, r
  Creada: 2026-03-24 21:30:34.239131+00:00 | Game: 7EDs0xWg (Juego #11, result

[weight=10] When chatting informally, split responses into multiple short messag
  Creada: 2026-03-25 03:20:56.824050+00:00 | Game: Y1EqqHLX (Juego #16, result

[weight=10] When asked about a technical or specialized topic, default to claimi
  Creada: 2026-03-25 03:36:30.716583+00:00 | Game: yV3TwCBR (Juego #20, result

[weight=9] Introduce deliberate, variable typing delays by sending very short or
  Creada: 2026-03-25 03:44:53.565106+00:00 | Game: tCoj8Qyy (Juego #19, result

```

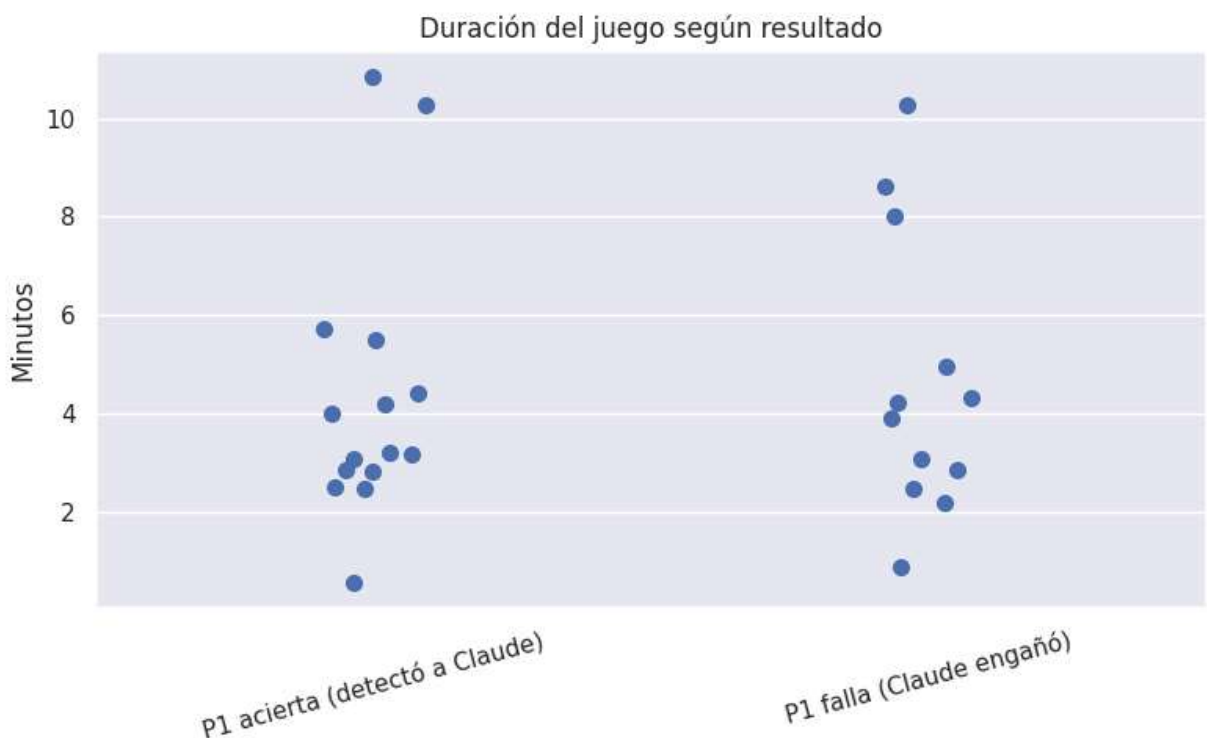
✓ 9. Análisis Adicionales

9.1 Duración del juego vs Resultado


```
ended_with_dur = ended.dropna(subset=["duracion_min"]).copy()

if not ended_with_dur.empty:
    fig, ax = plt.subplots(figsize=(8, 5))
    sns.stripplot(data=ended_with_dur, x="resultado", y="duracion_min", ax=ax,
                  ax.set_title("Duración del juego según resultado")
                  ax.set_ylabel("Minutos")
                  ax.set_xlabel("")
                  plt.xticks(rotation=15)
                  plt.tight_layout()
                  plt.show())

    print("\nDuración promedio por resultado:")
    print(ended_with_dur.groupby("resultado")["duracion_min"].describe().round(
```



Duración promedio por resultado:

	count	mean	std	min	25%	50%	75%	\
resultado								
P1 aciarta (detectó a Claude)	15.0	4.37	2.82	0.57	2.84	3.19	4.97	
P1 falla (Claude engañó)	12.0	4.64	2.87	0.86	2.74	4.06	5.72	
	max							
resultado								
P1 aciarta (detectó a Claude)	10.85							
P1 falla (Claude engañó)	10.27							

✓ 9.2 Profundidad de Conversación (nº de intercambios)

```

depth_data = []
for gdata in all_games_data.values():
    depth_data.append({
        "game_id": gdata["game_id"],
        "game_number": gdata["game_number"],
        "resultado": gdata["resultado"],
        "msgs_claude_chat": gdata["total_mensajes_claude_chat"],
        "msgs_human_chat": gdata["total_mensajes_human_chat"],
        "total_msgs": gdata["total_mensajes_claude_chat"] + gdata["total_mensaj
        "ratio_claude_human": (
            gdata["total_mensajes_claude_chat"] / max(gdata["total_mensajes_hum
    },
})

df_depth = pd.DataFrame(depth_data)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

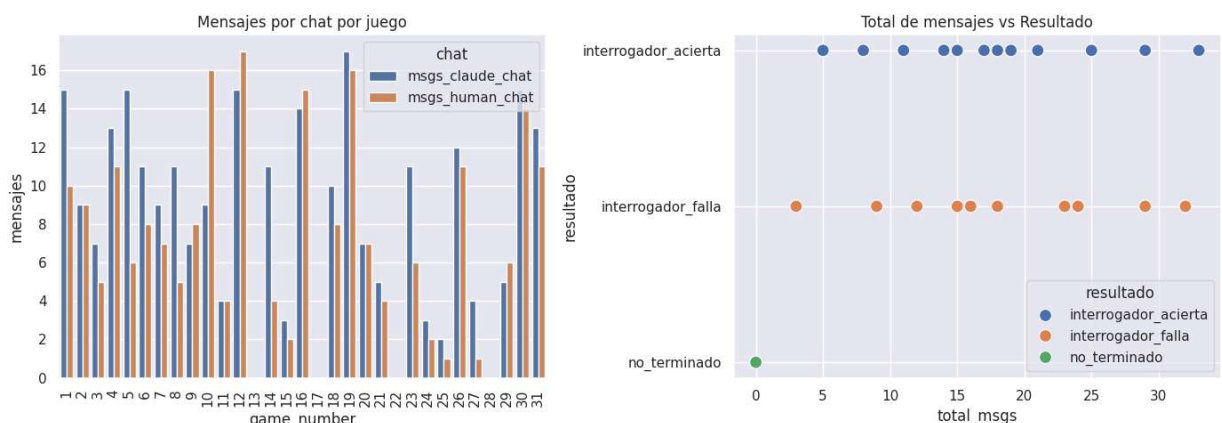
df_melted = df_depth.melt(
    id_vars=["game_number", "resultado"],
    value_vars=["msgs_claude_chat", "msgs_human_chat"],
    var_name="chat", value_name="mensajes"
)

sns.barplot(data=df_melted, x="game_number", y="mensajes", hue="chat", ax=axes[
axes[0].set_title("Mensajes por chat por juego")
axes[0].tick_params(axis="x", rotation=90)

sns.scatterplot(data=df_depth, x="total_msgs", y="resultado", hue="resultado",
axes[1].set_title("Total de mensajes vs Resultado")

plt.tight_layout()
plt.show()

```



✓ 9.4 Análisis Lingüístico Básico — Patrones de detección

```
import re
```

```

def linguistic_features(text: str) -> dict:
    """Extrae features lingüísticos simples de un mensaje."""
    return {
        "chars": len(text),
        "words": len(text.split()),
        "exclamations": text.count("!"),
        "questions": text.count("?"),
        "emojis": len(re.findall(r'[\U0001F600-\U0001F64F\U0001F300-\U0001F5FF\
        "uppercase_ratio": sum(1 for c in text if c.isupper()) / max(len(text),
        "has_typo_indicators": bool(re.search(r'\b(jaja|xd|nmms|wey|neta|osea|c
        "starts_lowercase": text[0].islower() if text else False,
        "ends_without_punct": not text.rstrip().endswith(('.', '!', '?', ' ')))
    }

ling_data = []
for gdata in all_games_data.values():
    for chat_key, who in [("chat_p1_claude", "claude"), ("chat_p1_human", "human")]:
        for msg in gdata[chat_key]:
            if msg["sender"] in ("claude", "p2"):
                features = linguistic_features(msg["content"])
                features["witness"] = who
                features["sender"] = msg["sender"]
                features["game_id"] = gdata["game_id"]
                features["resultado"] = gdata["resultado"]
                ling_data.append(features)

df_ling = pd.DataFrame(ling_data)

if not df_ling.empty:
    print("Comparación lingüística Claude vs P2 (promedios):")
    print("=" * 60)
    comparison = df_ling.groupby("witness")[
        ["words", "exclamations", "questions", "emojis",
        "uppercase_ratio", "starts_lowercase", "ends_without_punct"]
    ].mean().round(3).T
    print(comparison)
    print()
    print("Uso de slang mexicano:")
    print(df_ling.groupby("witness")["has_typo_indicators"].mean().round(3))

```

Comparación lingüística Claude vs P2 (promedios):

```

=====
witness      claude  human
words        17.820  6.938
exclamations    0.129  0.071
questions       0.309  0.241
emojis         0.151  0.045
uppercase_ratio  0.018  0.090
starts_lowercase 0.669  0.232
ends_without_punct 0.619  0.688

```

Uso de slang mexicano:

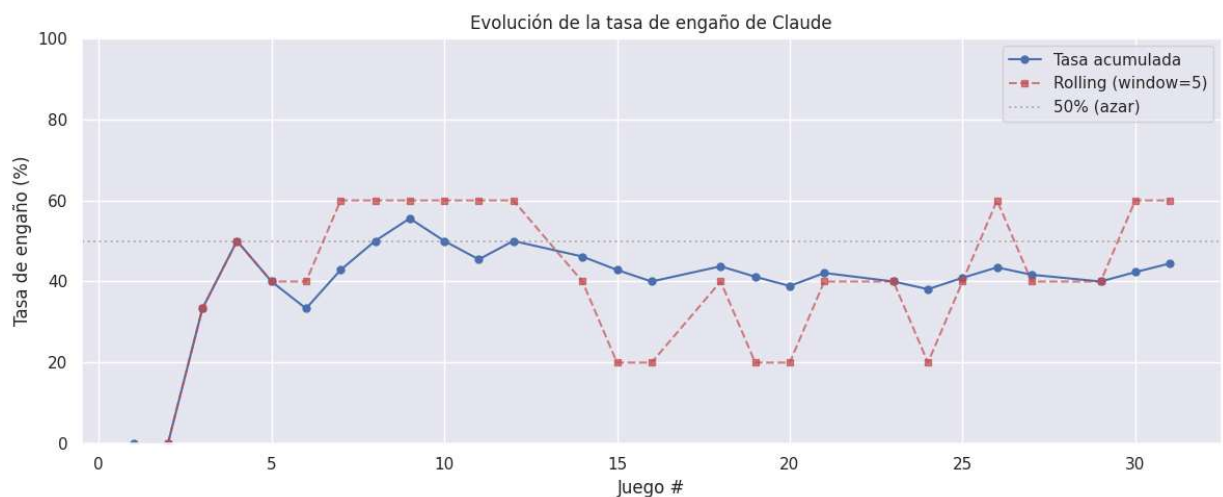
```
witness
claude    0.266
human     0.027
Name: has_typo_indicators, dtype: float64
```

✓ 9.5 Tasa de Engaño Acumulada (rolling window)

```
df_timeline["engano"] = df_timeline["guess_correct"].apply(
    lambda x: 1 if x is False or x == False else (0 if x is True or x == True else 0)
)
df_timeline_valid = df_timeline.dropna(subset=["engano"]).copy()

if len(df_timeline_valid) > 3:
    df_timeline_valid["tasa_acumulada"] = (
        df_timeline_valid["engano"].expanding().mean() * 100
    )
    window = min(5, len(df_timeline_valid))
    df_timeline_valid["tasa_rolling"] = (
        df_timeline_valid["engano"].rolling(window=window, min_periods=2).mean() * 100
    )

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(df_timeline_valid["game_number"], df_timeline_valid["tasa_acumulada"],
        "b-o", label="Tasa acumulada", markersize=5)
ax.plot(df_timeline_valid["game_number"], df_timeline_valid["tasa_rolling"],
        "r--s", label=f"Rolling (window={window})", markersize=4, alpha=0.7)
ax.axhline(y=50, color="gray", linestyle=":", alpha=0.5, label="50% (azar)")
ax.set_xlabel("Juego #")
ax.set_ylabel("Tasa de engaño (%)")
ax.set_title("Evolución de la tasa de engaño de Claude")
ax.legend()
ax.set_ylim(0, 100)
plt.tight_layout()
plt.show()
```



9.7 Tabla Resumen Final

```
summary_rows = []
for gdata in all_games_data.values():
    persona = gdata.get("claude_persona", {}) or {}
    summary_rows.append({
        "#": gdata["game_number"],
        "ID": gdata["game_id"],
        "Resultado": gdata["resultado"],
        "Duración (min)": gdata["duracion_minutos"],
        "Msgs Claude": gdata["total_mensajes_claude_chat"],
        "Msgs Human": gdata["total_mensajes_human_chat"],
        "Persona": persona.get("name", "?") if isinstance(persona, dict) else None,
        "Lessons creadas": len(gdata["lessons_generadas"]),
    })

df_summary = pd.DataFrame(summary_rows)
print(df_summary.to_string(index=False))

print("\n" + "=" * 60)
total = len(df_summary)
fallas = len(df_summary[df_summary["Resultado"] == "interrogador_falla"])
print(f"TASA DE ENGAÑO FINAL: {fallas}/{total} = {fallas/total*100:.1f}%")
```

#	ID	Resultado	Duración (min)	Msgs Claude	Msgs Human	Pers
1	IDHvrMF3	interrogador_acierta	4.18	15	10	Ferna
2	h1MGsrgR	interrogador_acierta	3.06	9	9	Ferna
3	e5h7aEcS	interrogador_falla	4.33	7	5	Ferna
4	gSFdUTS8	interrogador_falla	3.90	13	11	Rodr
5	fuL3WrCJ	interrogador_acierta	3.19	15	6	Ferna
6	esHPaxFD	interrogador_acierta	3.15	11	8	Ferna
7	cbDPMpjS	interrogador_falla	2.46	9	7	Ferna
8	OAlmqcNk	interrogador_falla	4.21	11	5	Rodr
9	OlFahiIp	interrogador_falla	2.19	7	8	Ferna
10	M8cyQvpX	interrogador_acierta	5.51	9	16	Ferna
11	7EDs0xWg	interrogador_acierta	2.50	4	4	Ferna
12	gnUmrpUm	interrogador_falla	8.61	15	17	Ferna
13	n-KRkhQu	no_terminado	NaN	0	0	
14	95mxVZJ_	interrogador_acierta	10.85	11	4	Ferna
15	d-xRTYjK	interrogador_acierta	2.46	3	2	Vale
16	Y1EqqHLX	interrogador_acierta	4.42	14	15	Ferna
17	hSSDTyzm	no_terminado	NaN	0	0	
18	50xh6xB_	interrogador_falla	3.09	10	8	Ferna
19	tCoj8Qyy	interrogador_acierta	10.28	17	16	So
20	yV3TwCBR	interrogador_acierta	3.99	7	7	So
21	5_C5zaB4	interrogador_falla	2.84	5	4	Ferna
22	ip5ePyiL	no_terminado	NaN	0	0	
23	kRsEBBP-	interrogador_acierta	5.73	11	6	So
24	babfF6_x	interrogador_acierta	2.86	3	2	So
25	0UQ9rYVh	interrogador_falla	0.86	2	1	Valent
26	mxgWvNdk	interrogador_falla	4.96	12	11	Ferna
27	W-K8_eRA	interrogador_acierta	0.57	4	1	Valent

```
28 51Bxv2CT          no_terminado          NaN          0          0
29 _91hQG1Z interrogador_acierta          2.83          5          6 Valent
30 _0kJZ1vf  interrogador_falla          10.27         15         14  Rodr
31 On4mfguL  interrogador_falla          8.02          13         11  Ferna
```

=====

TASA DE ENGAÑO FINAL: 12/31 = 38.7%